SPA con Vue 3 y GitHub Pages

Menos teoría.
Más interacción.

Documento de evidencias de cumplimiento. Aplicación educativa con cuatro rutas, ejemplos interactivos y publicación del build en GitHub Pages.

Repositorio de código fuente

https://github.com/juanvesta123/vue-estructuras-control

Sitio compilado y publicado

https://juanvesta123.github.io/vue-estructuras-control/

Autor: juanvesta123
Tecnologías: Vue 3, Vue Router, Vite y GitHub Actions.
Capturas reales del sitio público y de los archivos fuente del mismo proyecto.

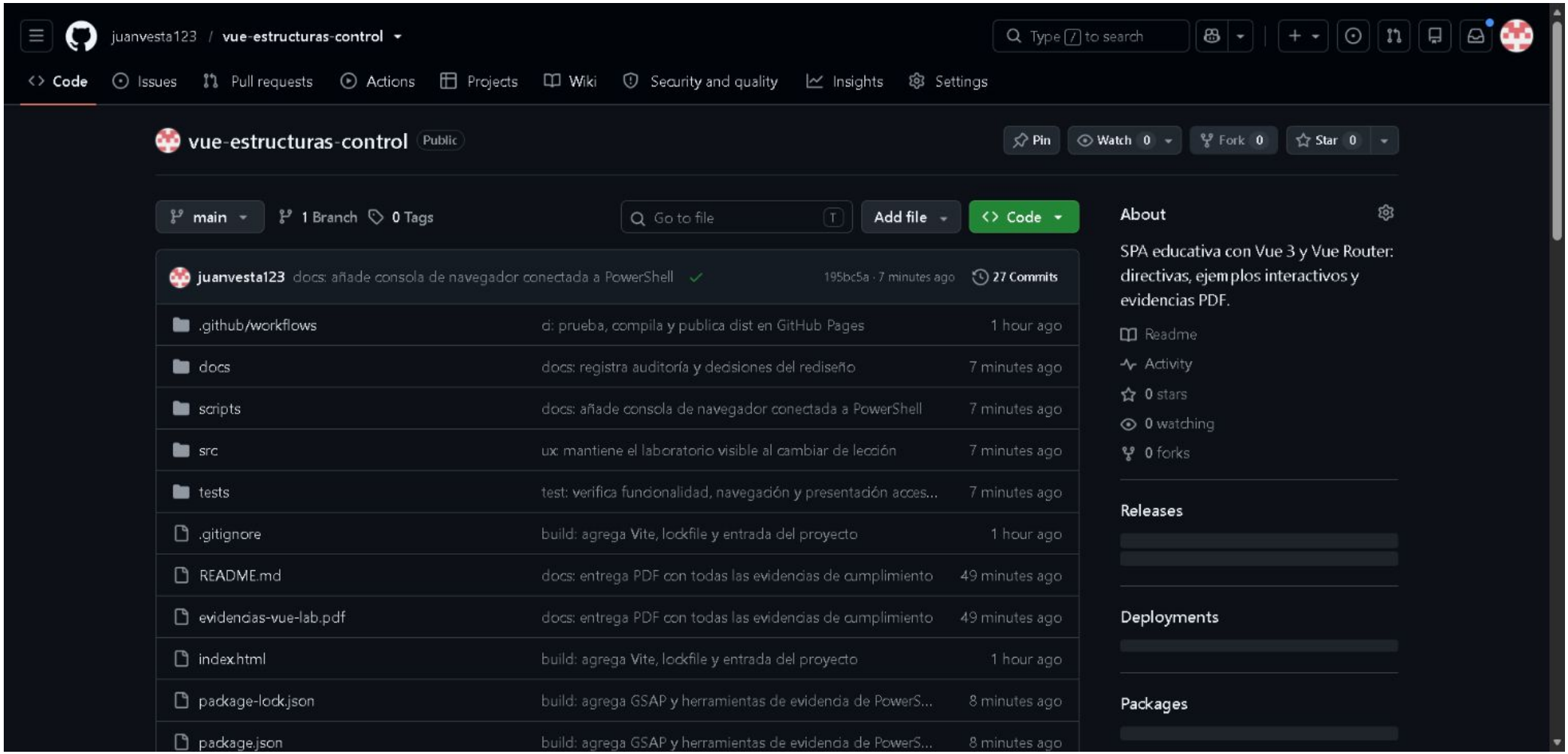
Guía de evidencias y especificaciones

01	Repositorio personal, código fuente y commits	Páginas 3-5
02	Build con npm run build y GitHub Pages	Páginas 6-7
03	SPA con cuatro rutas de Vue Router	Páginas 8-9
04	v-if / v-else-if / v-else	Páginas 10-13
05	v-show: visible y oculto	Páginas 14-16
06	v-for: lista reactiva	Páginas 17-18
07	v-bind:key dentro de v-for	Páginas 19-21
08	router-link y navegación SPA sin recarga	Páginas 22-24

Criterio de presentación: una sola captura por página. Cada evidencia agrupa capturas de interfaz y código, con un máximo de dos párrafos explicativos. Las capturas de código muestran la salida de comandos Get-Content ejecutados por procesos reales de Windows PowerShell y transmitidos a una terminal xterm.js en el navegador. No son visores de código ni marcos simulados de macOS.

Repositorio personal con código Vue

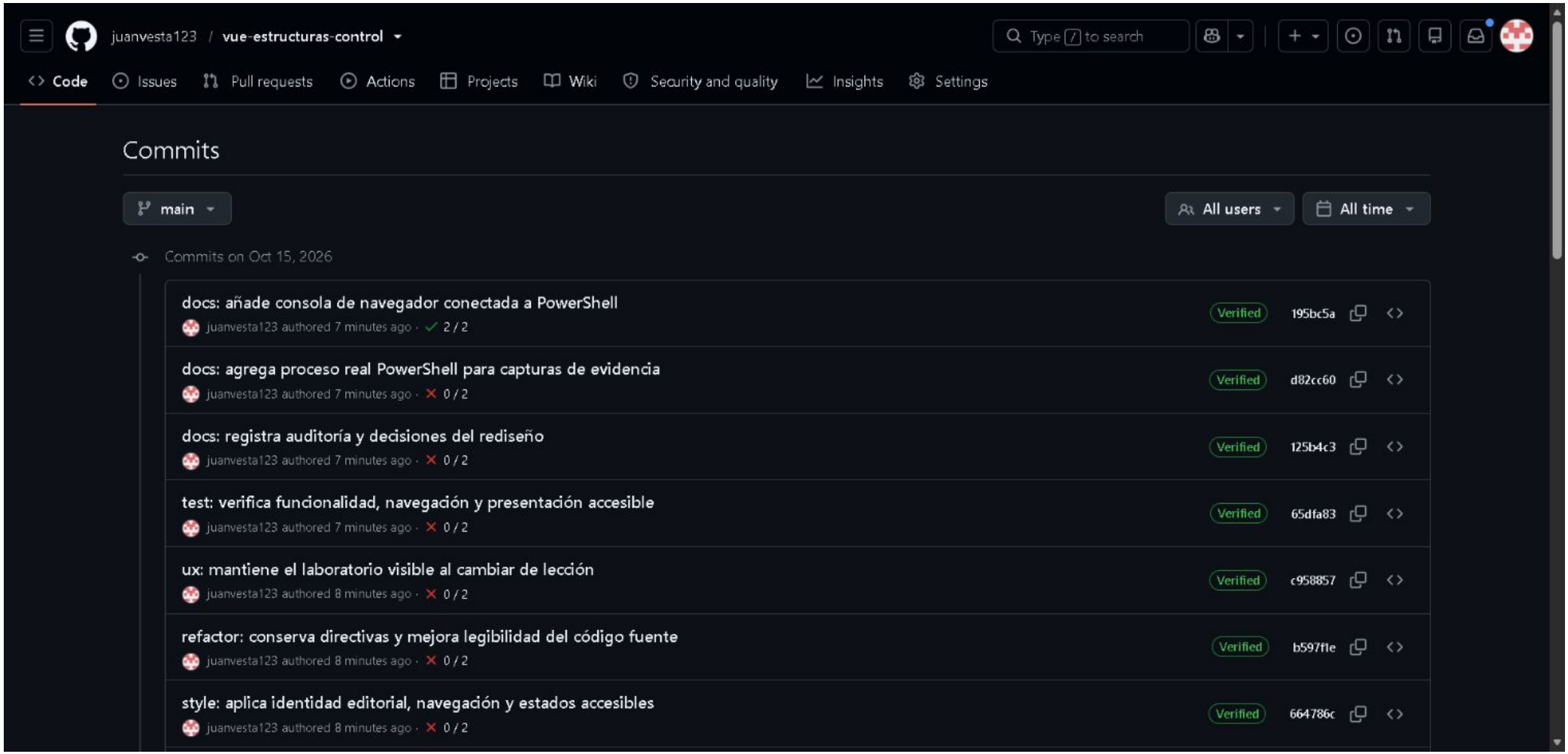
INTERFAZ / GITHUB



El repositorio público pertenece a juanvesta123 e incluye src, componentes, router, vistas, tests, configuración de Vite y dependencias con su lockfile. El código fuente está disponible para revisión y reproducción.

Commits durante el desarrollo

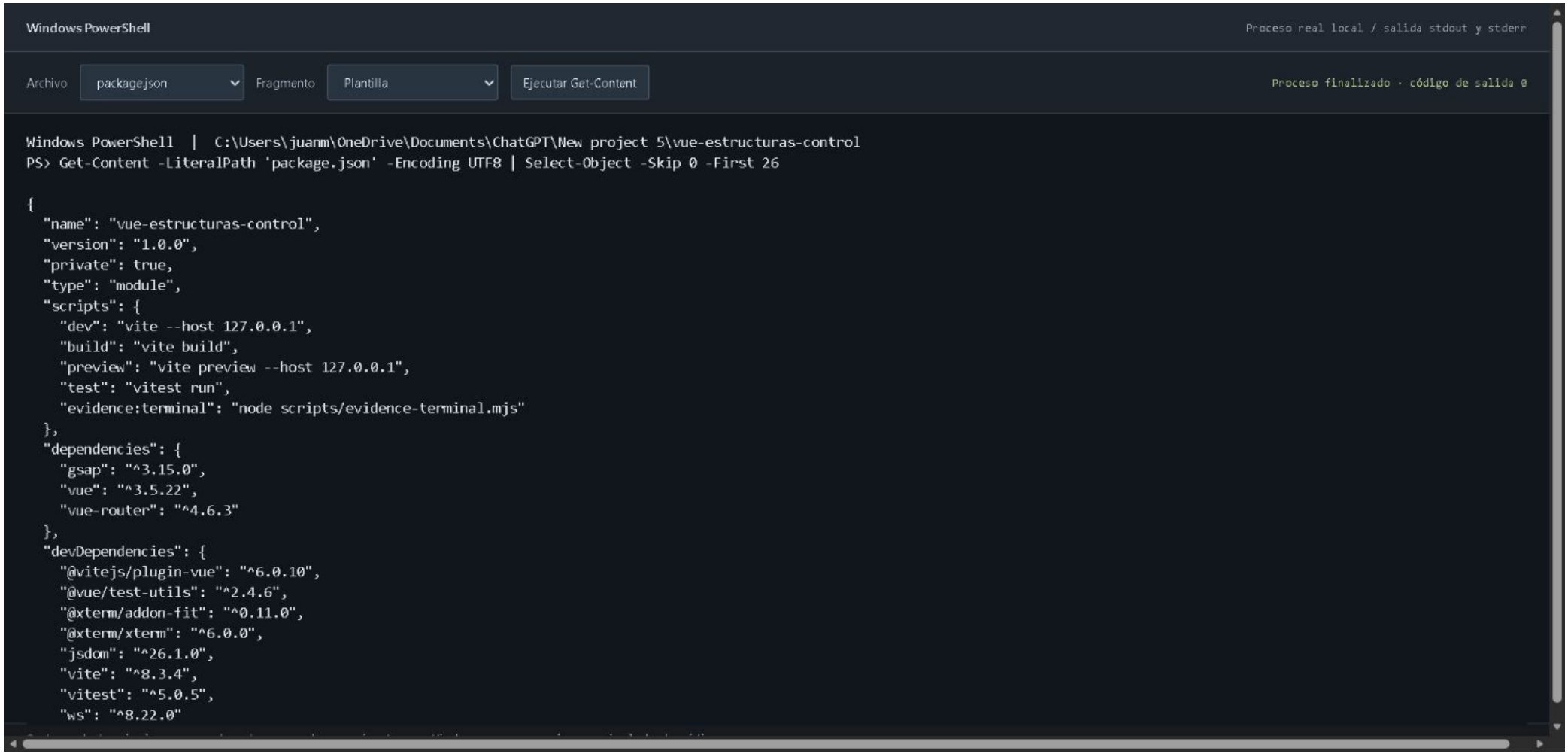
INTERFAZ / HISTORIAL DE GITHUB



Historial real: configuración, navegación, componentes, directivas, pruebas y publicación.

Dependencias y scripts del proyecto

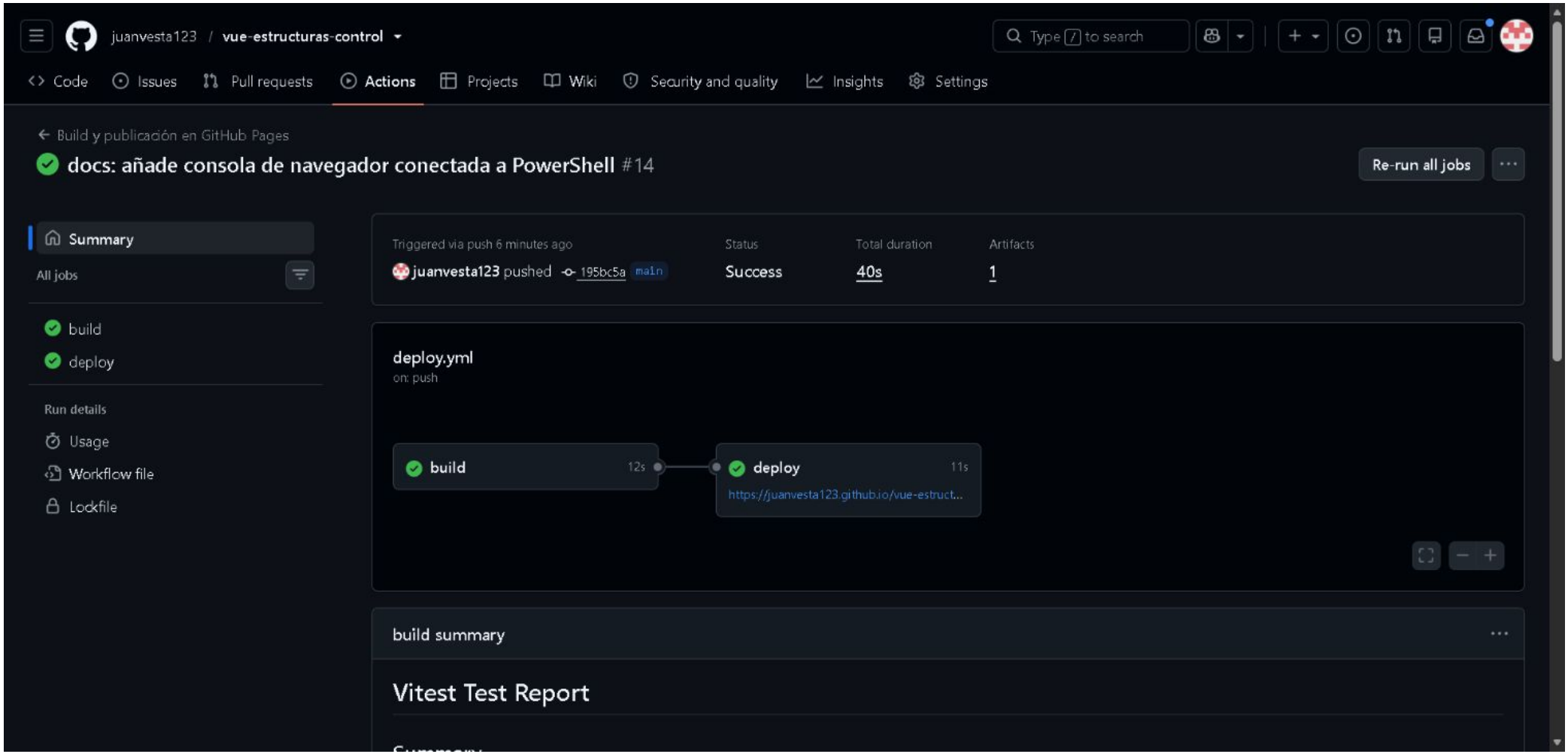
CÓDIGO FUENTE / package.json



package.json declara Vue 3 y Vue Router, además de los scripts dev, test, build y preview. El historial documenta avances separados durante el desarrollo, en lugar de una entrega sin commits intermedios.

Build y despliegue completados

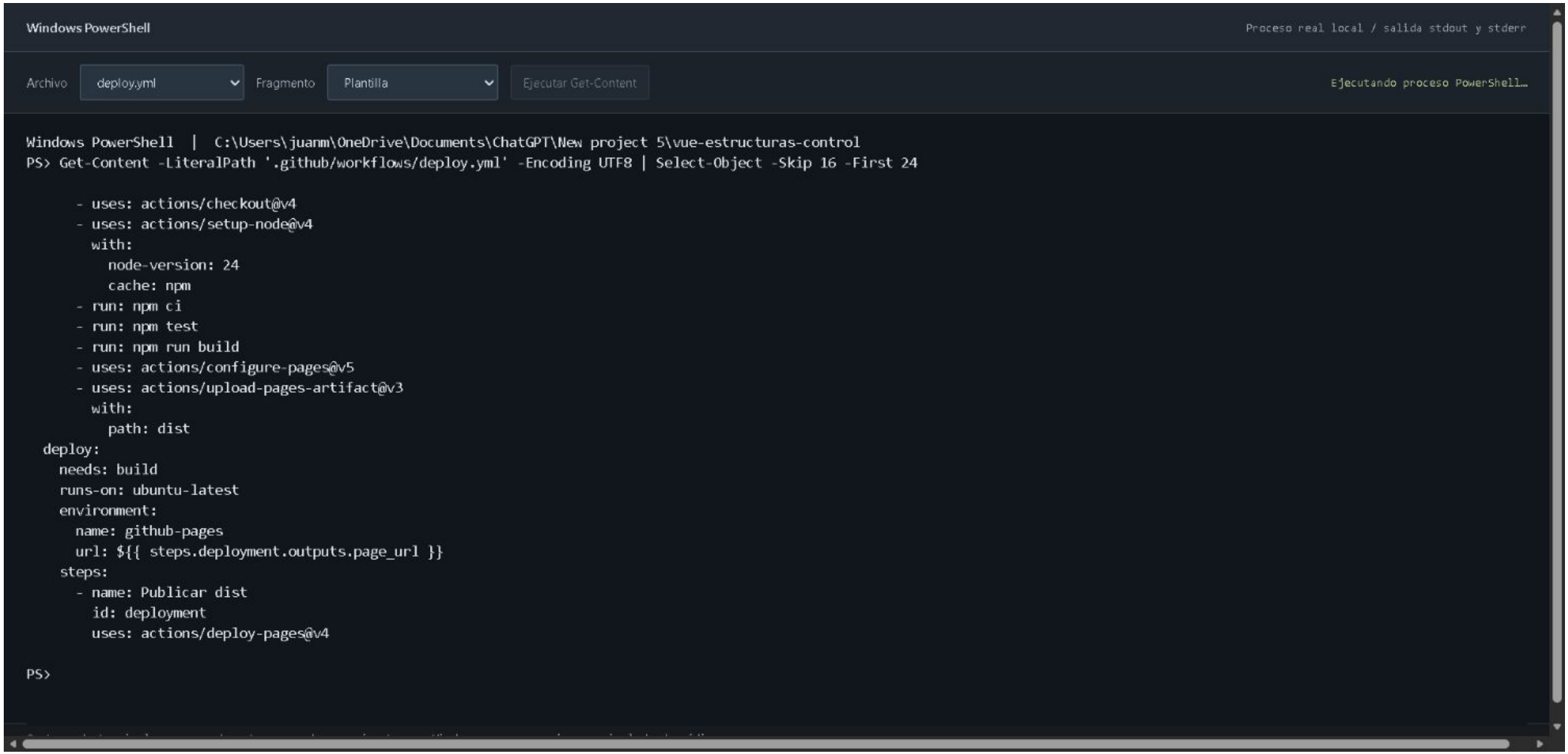
INTERFAZ / GITHUB ACTIONS



GitHub Actions completó los trabajos build y deploy. La ejecución verifica las pruebas, compila el proyecto y publica el artefacto github-pages. El sitio y sus archivos JavaScript y CSS respondieron HTTP 200 en la verificación pública.

Compilar y publicar únicamente dist

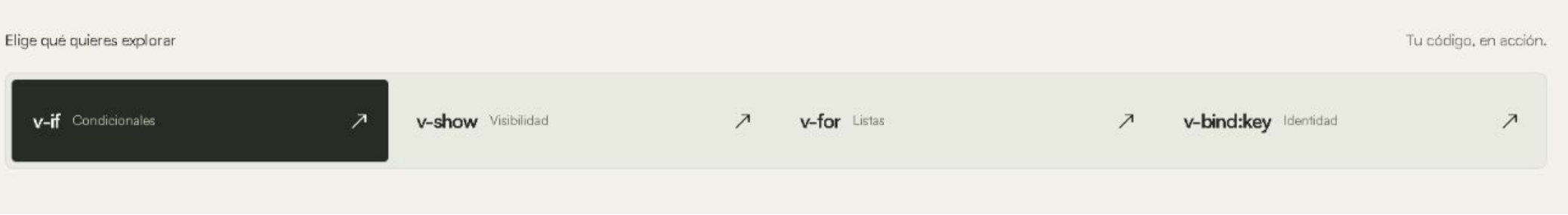
CÓDIGO FUENTE / .github/workflows/deploy.yml



El workflow ejecuta npm ci, npm test y npm run build; después sube la carpeta dist con upload-pages-artifact y la publica mediante deploy-pages. Pages usa GitHub Actions como origen. La URL entregada es la del sitio compilado, distinta de la del repositorio.

Cuatro lecciones accesibles desde el menú

INTERFAZ / SITIO PUBLICADO



El menú ofrece cuatro páginas dedicadas a condicionales, visibilidad, listas e identidad. Cada ruta presenta una explicación breve, un ejemplo de código y una demostración que aplica la directiva correspondiente.

Registro de rutas con Vue Router

CÓDIGO FUENTE / src/router/index.js

Windows PowerShell

Proceso real local / salida stdout y stderr

Archivo router/index.js Fragmento Plantilla Ejecutar Get-Content

Ejecutando proceso PowerShell...

```
import { createRouter, createWebHashHistory } from "vue-router";
import IfView from "../views/IfView.vue";
import ShowView from "../views/ShowView.vue";
import ForView from "../views/ForView.vue";
import KeyView from "../views/KeyView.vue";

export default createRouter({
  history: createWebHashHistory(import.meta.env.BASE_URL),
  routes: [
    { path: "/", redirect: "/v-if" },
    {
      path: "/v-if",
      component: IfView,
      meta: { title: "v-if · Condicionales" },
    },
    {
      path: "/v-show",
      component: ShowView,
      meta: { title: "v-show · Visibilidad" },
    },
    { path: "/v-for", component: ForView, meta: { title: "v-for · Listas" } },
    {
      path: "/v-bind-key",
      component: KeyView,
      meta: { title: "v-bind:key · Identidad" },
    },

PS>
```

createRouter registra /v-if, /v-show, /v-for y /v-bind-key con componentes independientes. createWebHashHistory permite acceder directamente o recargar una ruta en GitHub Pages; la ruta inicial redirige a /v-if.

v-else-if: calificación aprobatoria

INTERFAZ / #/v-if / 85 PUNTOS

Decide qué mostrar.

Pruébalo en vivo

v-if renderiza un elemento cuando su condición es verdadera. v-else-if comprueba otra condición y v-else cubre el resto de los casos. Solo una rama aparece en el DOM.

</> IfView.vue — Vue

```
1  const calificacion = ref(85)
2
3  <input v-model.number="calificacion"
4         type="range" min="0" max="100" />
5
6  <p v-if="calificacion >= 90">
7    Excelente
8  </p>
9  <p v-else-if="calificacion >= 70">
10   Aprobado
11 </p>
12 <p v-else>Sigue practicando</p>
```

↳ Modifica los datos. Observa el resultado.

Vista previa — Reactividad en tiempo real

¿Cómo te fue en el examen?

Mueve el control y observa la rama activa.

85 / 100 puntos

Calificación

0 — 70 · Aprobado — 100

Aprobado

`Se cumple: calificacion >= 70`

La calificación reactiva controla tres ramas contiguas. Con 85 se muestra Aprobado; con 100, Excelente; y con 0, Sigue practicando. Las capturas siguientes muestran los tres estados reales de la misma ruta.

v-if: calificación excelente

INTERFAZ / #/v-if / 100 PUNTOS

Decide qué mostrar.

Pruébalo en vivo

v-if renderiza un elemento cuando su condición es verdadera. v-else-if comprueba otra condición y v-else cubre el resto de los casos. Solo una rama aparece en el DOM.

</> IfView.vue — Vue

```vue
const calificacion = ref(85)

<input v-model.number="calificacion"
       type="range" min="0" max="100" />

<p v-if="calificacion >= 90">
  Excelente
</p>
<p v-else-if="calificacion >= 70">
  Aprobado
</p>
<p v-else>Sigue practicando</p>
```

↳ Modifica los datos. Observa el resultado.

Vista previa — Reactividad en tiempo real

¿Cómo te fue en el examen?

Mueve el control y observa la rama activa.

100 / 100 puntos

Calificación

0 — 70 · Aprobado — 100

Excelente

`Se cumple: calificacion >= 90`

v-else: caso restante

INTERFAZ / #/v-if / 0 PUNTOS

Decide qué mostrar.

Pruébalo en vivo

v-if renderiza un elemento cuando su condición es verdadera. v-else-if comprueba otra condición y v-else cubre el resto de los casos. Solo una rama aparece en el DOM.

</> IfView.vue — Vue

```
1  const calificacion = ref(85)
2
3  <input v-model.number="calificacion"
4         type="range" min="0" max="100" />
5
6  <p v-if="calificacion >= 90">
7    Excelente
8  </p>
9  <p v-else-if="calificacion >= 70">
10   Aprobado
11 </p>
12 <p v-else>Sigue practicando</p>
```

↳ Modifica los datos. Observa el resultado.

Vista previa — Reactividad en tiempo real

¿Cómo te fue en el examen?

Mueve el control y observa la rama activa.

0 / 100 puntos

Calificación

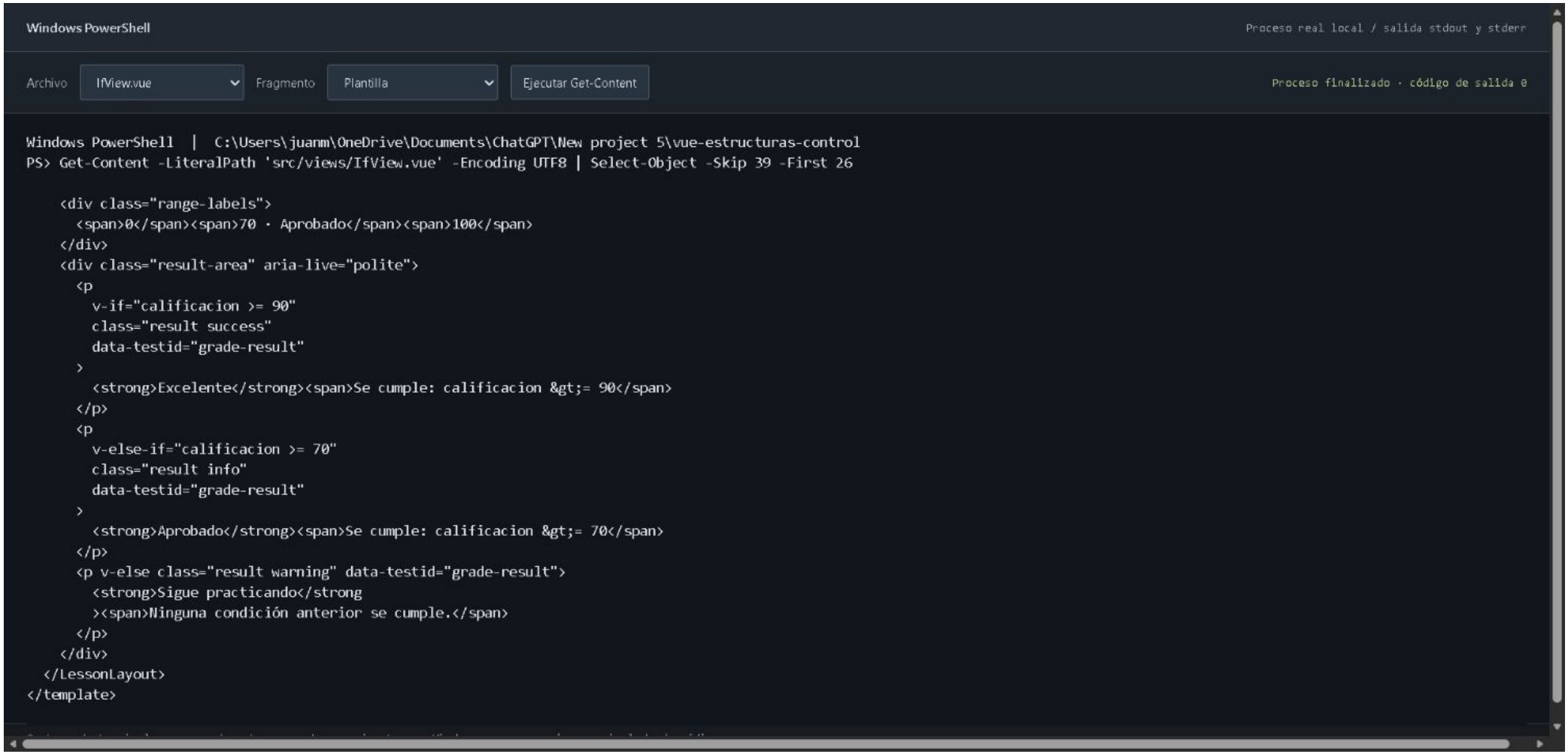
0 — 70 · Aprobado — 100

Sigue practicando

Ninguna condición anterior se cumple.

Las tres ramas en el código fuente

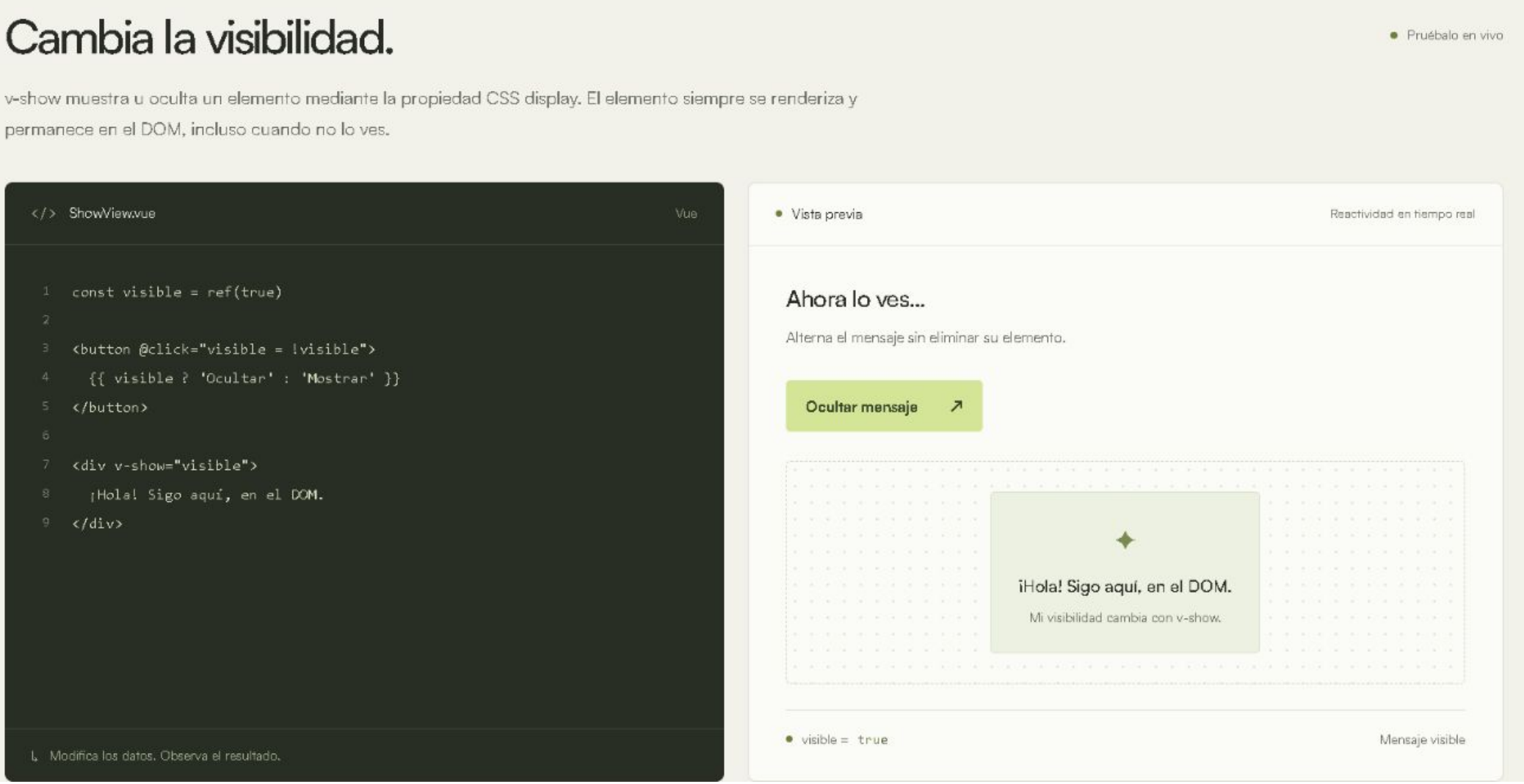
CÓDIGO FUENTE / src/views/IfView.vue



v-model.number actualiza calificacion. v-if comprueba >= 90; v-else-if comprueba >= 70; v-else cubre los valores restantes. Solo se renderiza la rama correspondiente. Las pruebas incluyen los límites 90, 89, 70 y 69.

v-show: mensaje visible

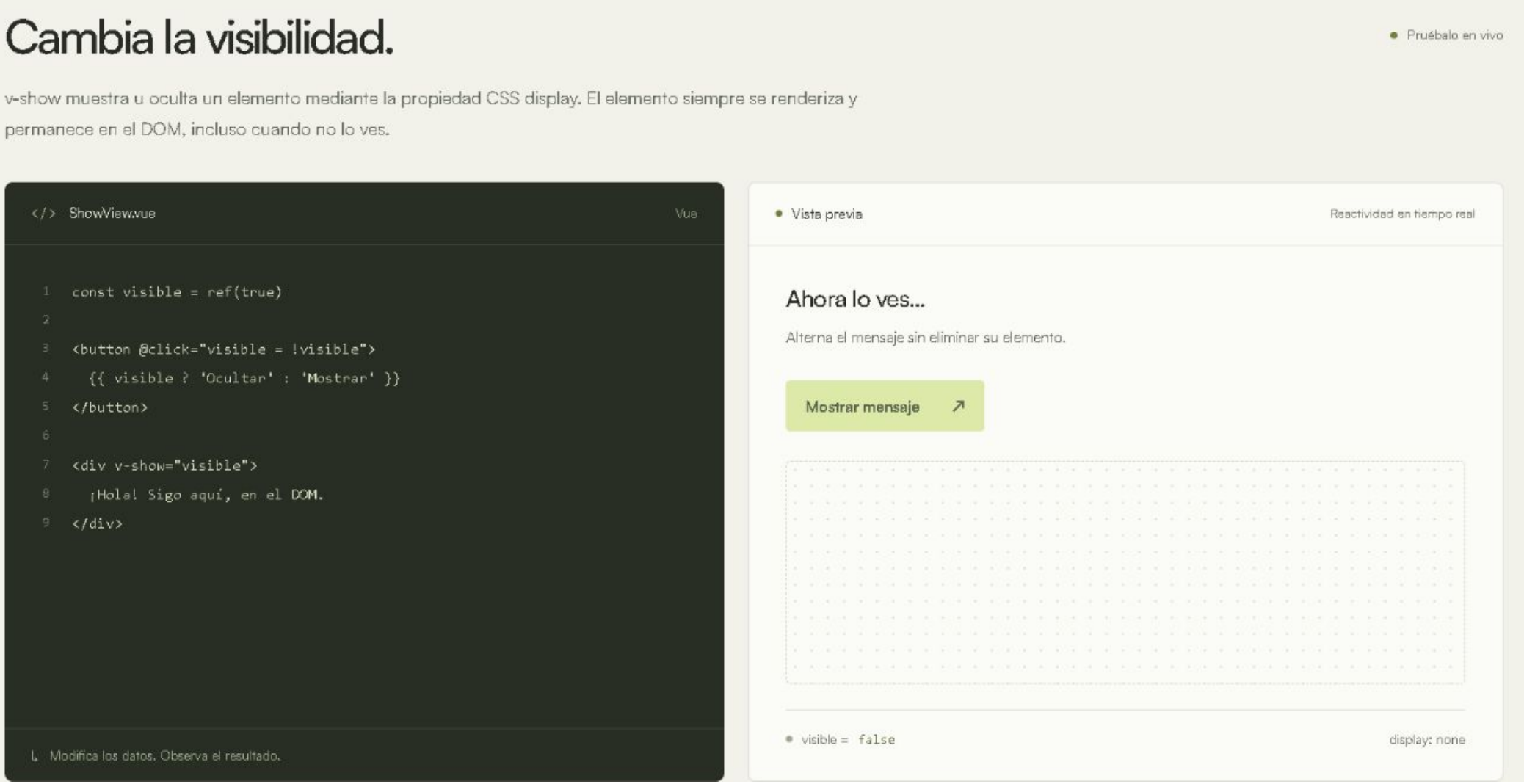
INTERFAZ / #/v-show / visible = true



El botón alterna el valor booleano visible. Cuando es true aparece el mensaje; cuando es false deja de verse, pero su elemento sigue presente en el DOM.

v-show: mensaje oculto

INTERFAZ / #/v-show / visible = false



Estado comprobado en el DOM publicado: style="display: none;".

Mostrar y ocultar sin retirar el nodo

CÓDIGO FUENTE / src/views/ShowView.vue

Windows PowerShell

Proceso real local / salida stdout y stderr

Archivo ShowView.vue Fragmento Plantilla Ejecutar Get-Content

Ejecutando proceso PowerShell…

```
Windows PowerShell  |  C:\Users\juanm\OneDrive\Documents\ChatGPT\New project 5\vue-estructuras-control
PS> Get-Content -LiteralPath 'src/views/ShowView.vue' -Encoding UTF8 | Select-Object -Skip 30 -First 26

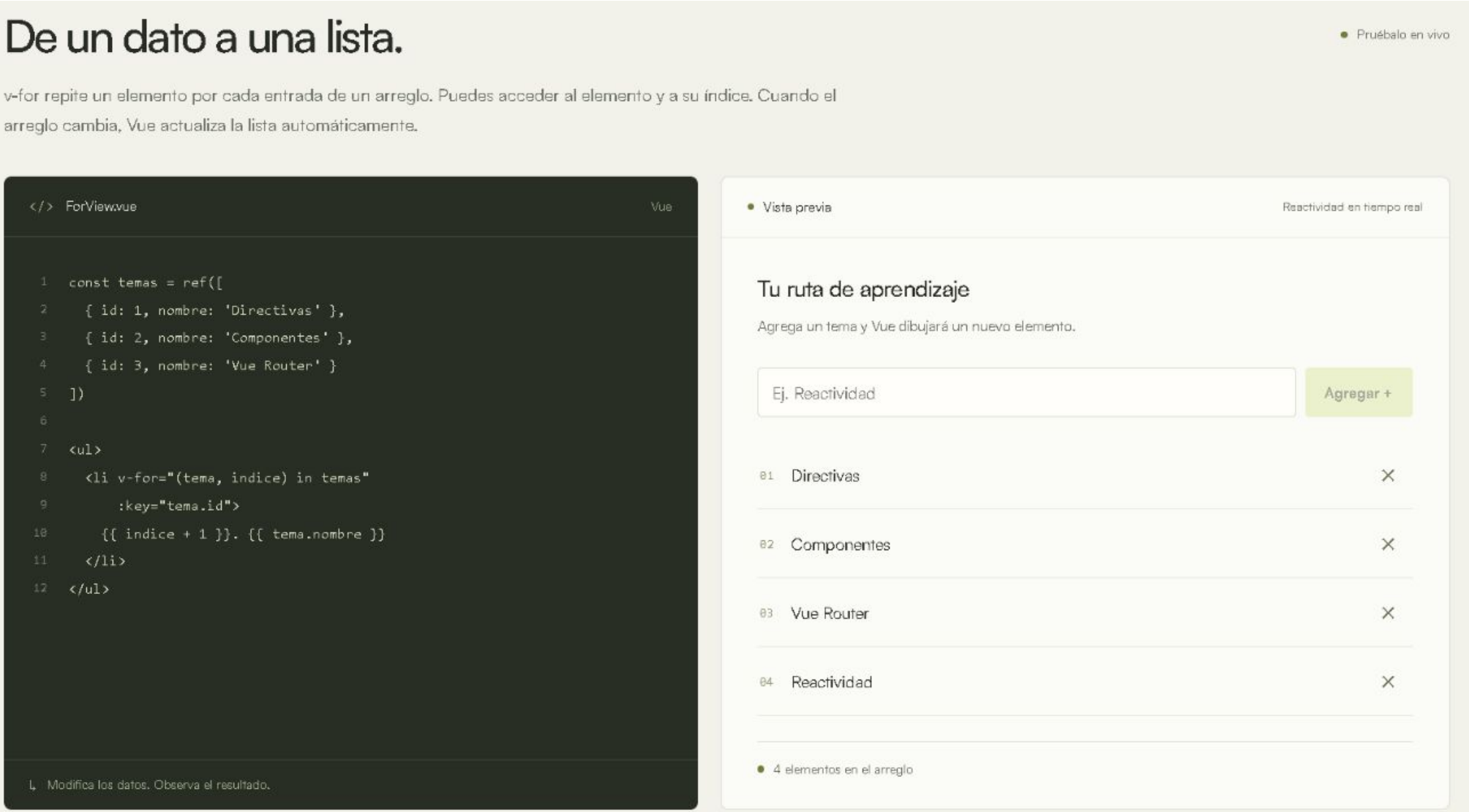
      {{ visible ? "Ocultar mensaje" : "Mostrar mensaje" }} <span>↗</span>
    </button>
    <div class="visibility-stage">
      <div
        id="mensaje"
        v-show="visible"
        class="message-card"
        data-testid="secret"
      >
        <span class="message-icon">✦</span
        ><strong>¡Hola! Sigo aquí, en el DOM.</strong>
        <p>Mi visibilidad cambia con v-show.</p>
      </div>
    </div>
    <p class="state-line" aria-live="polite">
      <i class="dot" :class="{ inactive: !visible }"></i> visible =
      <code>{{ visible }}</code
      ><span class="state-detail">{{
        visible ? "Mensaje visible" : "display: none"
      }}</span>
    </p>
  </LessonLayout>
</template>

PS>
```

La directiva v-show="visible" modifica display. La prueba automatizada compara la identidad del nodo antes y después de ocultarlo, y comprueba que al volver a mostrarlo se restaura su visibilidad.

v-for: agregar un tema a la lista

INTERFAZ / #/v-for / 4 ELEMENTOS



La lista inicia con tres temas. Al escribir Reactividad y pulsar Agregar aparece un cuarto elemento sin actualizar manualmente el HTML. Cada fila permite eliminar su tema y la cantidad se actualiza.

Iteración sobre un arreglo reactivo

CÓDIGO FUENTE / src/views/ForView.vue

Windows PowerShell

Proceso real local / salida stdout y stderr

Archivo ForView.vue Fragmento Plantilla Ejecutar Get-Content

Ejecutando proceso PowerShell…

```
PS> Get-Content -LiteralPath 'src/views/ForView.vue' -Encoding UTF8 | Select-Object -Skip 50 -First 26

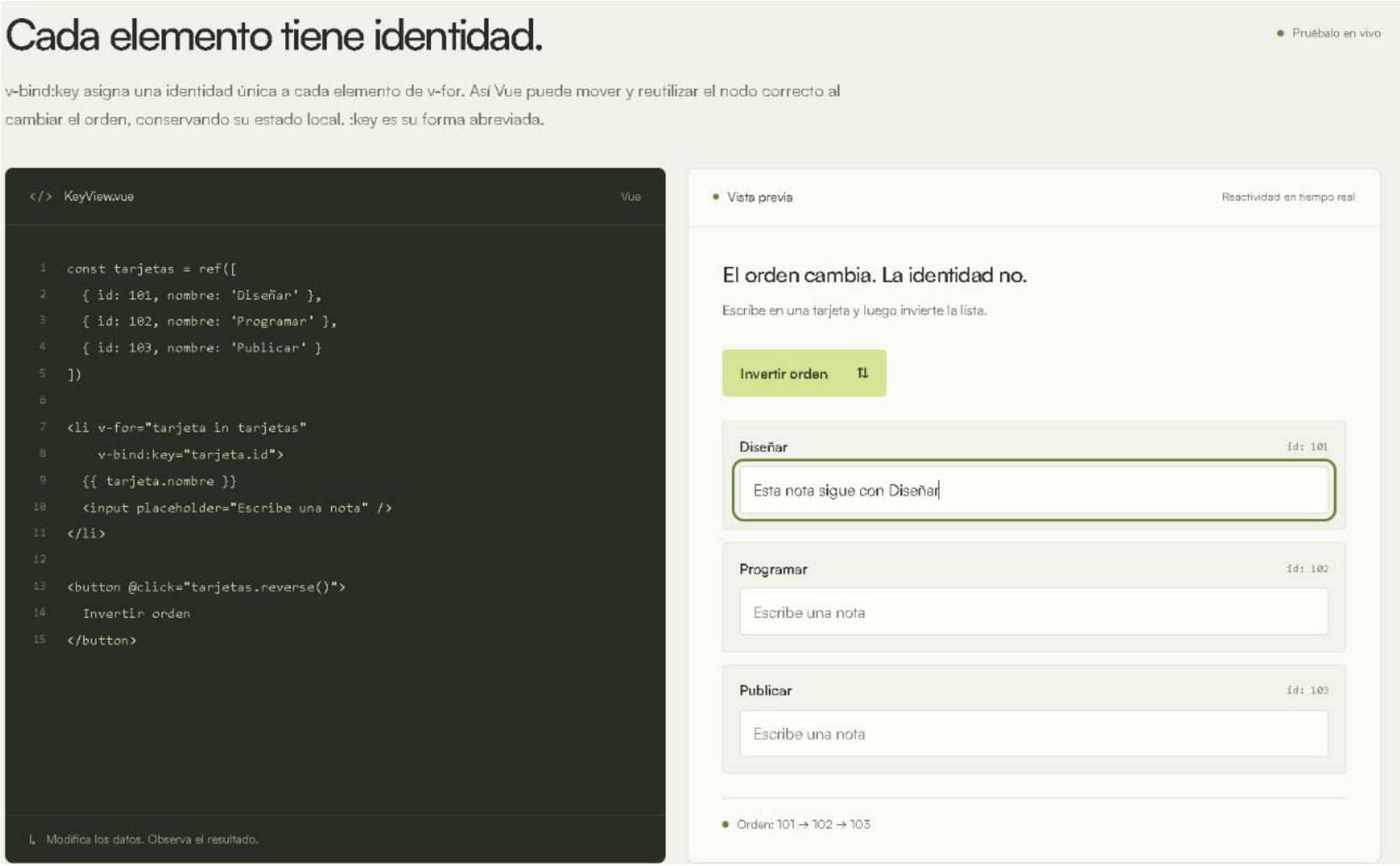
      >
        Agregar +
      </button>
    </form>
    <ul class="topic-list">
      <li v-for="(tema, indice) in temas" :key="tema.id" data-testid="topic">
        <span class="item-index">{{ String(indice + 1).padStart(2, "0") }}</span
        ><span>{{ tema.nombre }}</span
        ><button
          class="remove-button"
          @click="temas = temas.filter((t) => t.id !== tema.id)"
          :aria-label="`Eliminar ${tema.nombre}`"
        >
          ×
        </button>
      </li>
    </ul>
    <p v-if="!temas.length" class="muted">
      La lista está vacía. Agrega tu primer tema.
    </p>
    <p class="state-line" aria-live="polite">
      <i class="dot"></i> {{ temas.length }} elementos en el arreglo
    </p>
  </LessonLayout>
</template>

PS>
```

v-for="(tema, indice) in temas" crea una fila por tema. El índice numera las filas y :key="tema.id" identifica cada elemento. agregarTema rechaza entradas vacías o con espacios y asigna un identificador nuevo y estable.

v-bind:key: nota antes de reordenar

INTERFAZ / #/v-bind-key / ORDEN 101, 102, 103



Se escribe una nota en Diseñar, cuya identidad es id 101. Al invertir la lista, Diseñar pasa al final y su nota permanece con esa tarjeta, demostrando la reutilización correcta del nodo.

La nota se conserva después de invertir

INTERFAZ / #/v-bind-key / ORDEN 103, 102, 101



Identificador estable dentro del bucle

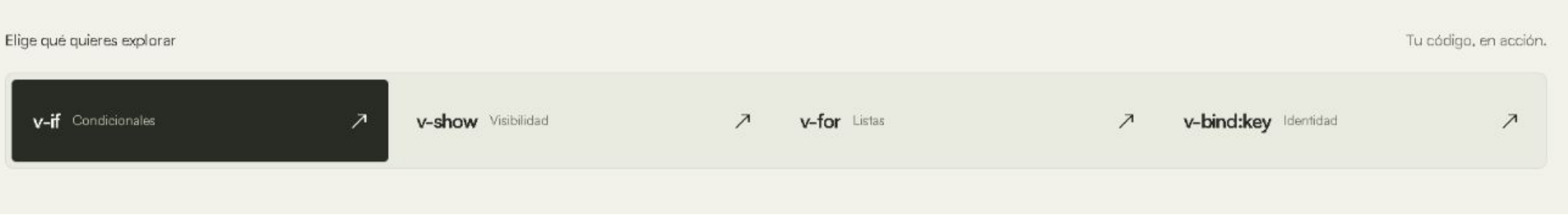
CÓDIGO FUENTE / src/views/KeyView.vue



El li utiliza v-for="tarjeta in tarjetas" junto con v-bind:key="tarjeta.id". No se usa el índice como key. La prueba verifica el nuevo orden, el mismo nodo input para id 101 y la conservación de su valor.

Menú que cambia de vista sin recarga

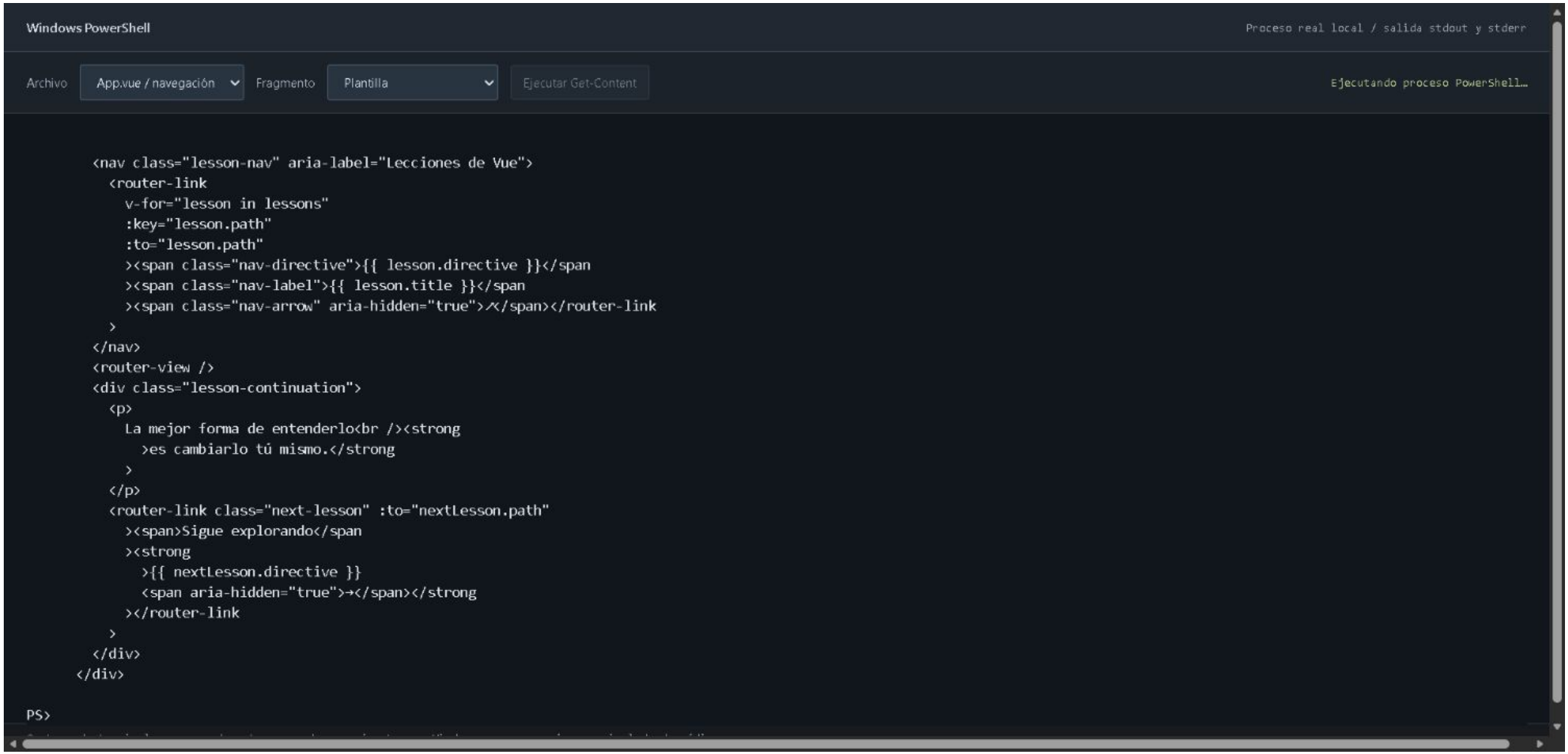
INTERFAZ / NAVEGACIÓN ENTRE LECCIONES



Los cuatro enlaces se probaron en el sitio publicado y cambian la vista mediante Vue Router. La navegación mantiene la estructura de la aplicación; una prueba adicional comprueba que cabecera y menú siguen siendo los mismos nodos al recorrer las cuatro rutas.

router-link y router-view

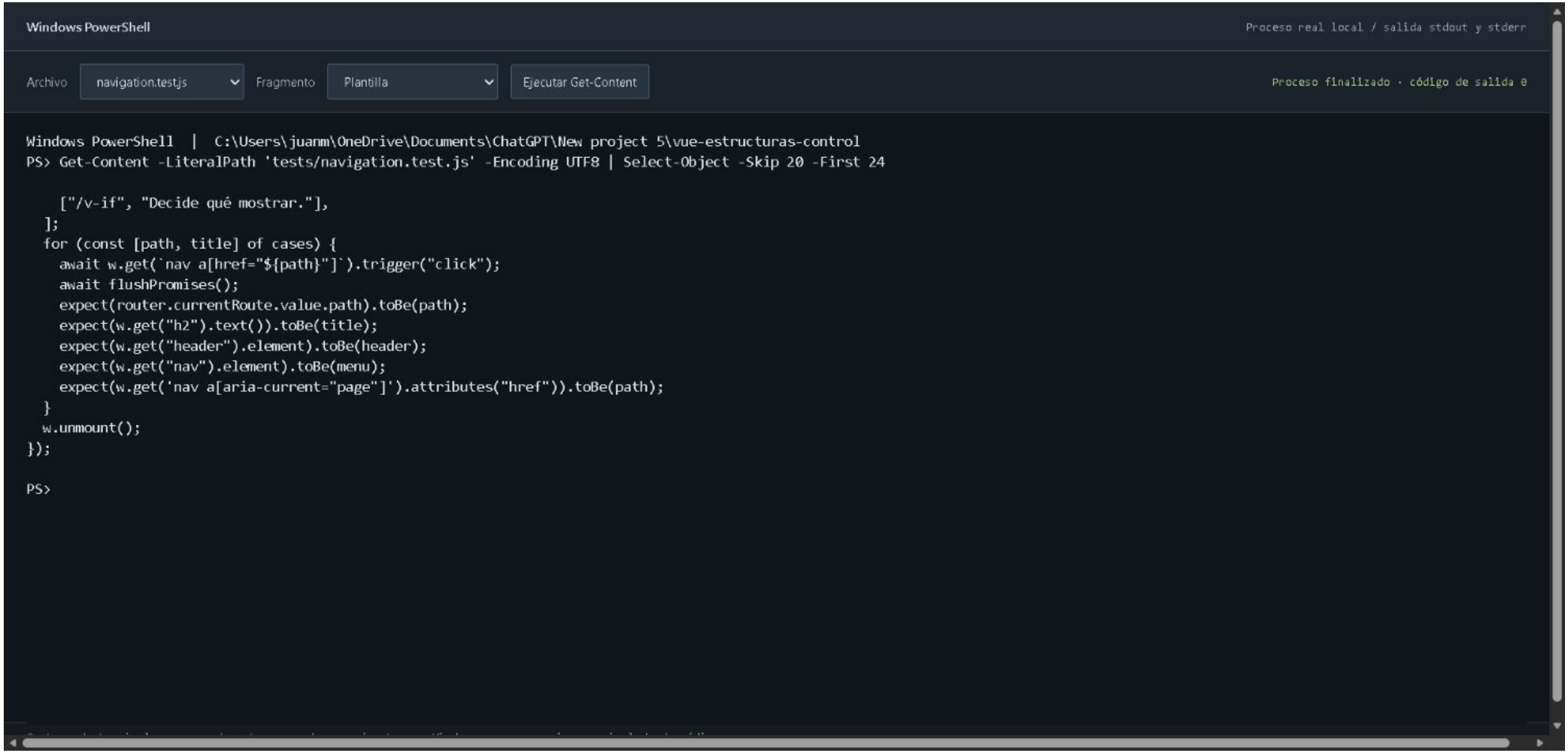
CÓDIGO FUENTE / src/App.vue



Los enlaces declaran destinos de Vue Router; router-view muestra la vista de la ruta actual.

Prueba de conservación de la aplicación

CÓDIGO FUENTE / tests/navigation.test.js



La prueba pulsa los enlaces, verifica la ruta y el título de cada lección, y compara los nodos de header y nav con sus referencias originales. Resultado final: 7 pruebas aprobadas en 3 archivos. El sitio también se comprobó a 320 y 390 px sin desbordamiento horizontal.

Cumplimiento final y enlaces de entrega

- **OK** Repositorio público con código fuente Vue y commits de desarrollo.
- **OK** Sitio en GitHub Pages a partir de npm run build y carpeta dist.
- **OK** SPA con cuatro rutas registradas en Vue Router.
- **OK** Ejemplo funcional de v-if / v-else-if / v-else.
- **OK** Ejemplo funcional de v-show.
- **OK** Ejemplo funcional de v-for.
- **OK** v-bind:key estable dentro de un bucle.
- **OK** Menú con router-link y contenido con router-view.
- **OK** Navegación SPA verificada y recarga directa de ruta funcional.
- **OK** PDF con capturas de interfaz y código, una captura por página.

Repositorio: https://github.com/juanvesta123/vue-estructuras-control
Sitio compilado: https://juanvesta123.github.io/vue-estructuras-control/
Validación: npm test: 7/7; npm run build: correcto; npm audit: 0 vulnerabilidades.

Todas las capturas de interfaz de las lecciones provienen del sitio público. Las capturas de terminal muestran comandos y salida real de PowerShell coincidentes con los archivos del repositorio. Este PDF se entrega como archivo independiente y también se agrega al repositorio.